

小结

447

关联容器支持通过关键字高效查找和提取元素。对关键字的使用将关联容器和顺序容器区分开来，顺序容器中是通过位置访问元素的。

标准库定义了 8 个关联容器，每个容器

- 是一个 `map` 或是一个 `set`。`map` 保存关键字-值对；`set` 只保存关键字。
- 要求关键字唯一或不要求。
- 保持关键字有序或不保证有序。

有序容器使用比较函数来比较关键字，从而将元素按顺序存储。默认情况下，比较操作是采用关键字类型的 `<` 运算符。无序容器使用关键字类型的 `==` 运算符和一个 `hash<key_type>` 类型的对象来组织元素。

允许重复关键字的容器的名字中都包含 `multi`；而使用哈希技术的容器的名字都以 `unordered` 开头。例如，`set` 是一个有序集合，其中每个关键字只可以出现一次；`unordered_multiset` 则是一个无序的关键字集合，其中关键字可以出现多次。

关联容器和顺序容器有很多共同的元素。但是，关联容器定义了一些新操作，并对一些和顺序容器和关联容器都支持的操作重新定义了含义或返回类型。操作的不同反映出关联容器使用关键字的特点。

有序容器的迭代器通过关键字有序访问容器中的元素。无论在有序容器中还是在无序容器中，具有相同关键字的元素都是相邻存储的。

术语表

关联数组 (associative array) 元素通过关键字而不是位置来索引的数组。我们称这样的数组将一个关键字映射到其关联的值。

关联容器 (associative container) 类型，保存对象的集合，支持通过关键字的高效查找。

hash 特殊的标准库模板，无序容器用它来管理元素的位置。

哈希函数 (hash function) 将给定类型的值映射到整形 (`size_t`) 值的函数。相等的值必须映射到相同的整数；不相等的值应尽可能映射到不同整数。

key_type 关联容器定义的类型，用来保存和提取值的关键字的类型。对于一个 `map`，`key_type` 是用来索引 `map` 的类型。对于 `set`，`key_type` 和 `value_type` 是一样的。

map 关联容器类型，定义了一个关联数组。类似 `vector`，`map` 是一个类模板。但是，一个 `map` 要用两个类型来定义：关键字的类型和关联的值的类型。在一个 `map` 中，一个给定关键字只能出现一次。每个关键字关联一个特定的值。解引用一个 `map` 迭代器会生成一个 `pair`，它保存一个 `const` 关键字及其关联的值。

mapped_type 映射类型定义的类型，就是映射中关键字关联的值的类型。

multimap 关联容器类型，类似 `map`，不同之处在于，在一个 `multimap` 中，一个给定的关键字可以出现多次。`multimap` 不支持下标操作。

multiset 保存关键字的关联容器类型。在一个 `multiset` 中，一个给定关键字可以出现多次。

448

pair 类型，保存名为 `first` 和 `second` 的 `public` 数据成员。`pair` 类型是模板类型，接受两个类型参数，作为其成员的类型。

set 保存关键字的关联容器。在一个 `set` 中，一个给定的关键字只能出现一次。

严格弱序 (strict weak ordering) 关联容器所使用的关键字间的关系。在一个严格弱序中，可以比较任意两个值并确定哪个更小。若任何一个都不小于另一个，则认为两个值相等。

无序容器 (unordered container) 关联容器，用哈希技术而不是比较操作来存储和访问元素。这类容器的性能依赖于哈希函数的质量。

unordered_map 保存关键字-值对的容器，不允许重复关键字。

unordered_multimap 保存关键字-值对的容器，允许重复关键字。

unordered_multiset 保存关键字的容器，

允许重复关键字。

unordered_set 保存关键字的容器，不允许重复关键字。

value_type 容器中元素的类型。对于 `set` 和 `multiset`，`value_type` 和 `key_type` 是一样的。对于 `map` 和 `multimap`，此类型是一个 `pair`，其 `first` 成员类型为 `const key_type`，`second` 成员类型为 `mapped_type`。

***运算符** 解引用运算符。当应用于 `map`、`set`、`multimap` 或 `multiset` 的迭代器时，会生成一个 `value_type` 值。注意，对 `map` 和 `multimap`，`value_type` 是一个 `pair`。

[]运算符 下标运算符。只能用于 `map` 和 `unordered_map` 类型的非 `const` 对象。对于映射类型，`[]` 接受一个索引，必须是一个 `key_type` 值（或者是能转换为 `key_type` 的类型）。生成一个 `mapped_type` 值。

第 12 章

动态内存

内容

12.1 动态内存与智能指针.....	400
12.2 动态数组.....	423
12.3 使用标准库：文本查询程序.....	430
小结.....	436
术语表.....	436

到目前为止，我们编写的程序中所使用的对象都有着严格定义的生存期。全局对象在程序启动时分配，在程序结束时销毁。对于局部自动对象，当我们进入其定义所在的程序块时被创建，在离开块时销毁。局部 `static` 对象在第一次使用前分配，在程序结束时销毁。

除了自动和 `static` 对象外，C++ 还支持动态分配对象。动态分配的对象的生存期与它们在哪里创建是无关的，只有当显式地被释放时，这些对象才会销毁。

动态对象的正确释放被证明是编程中极其容易出错的地方。为了更安全地使用动态对象，标准库定义了两个智能指针类型来管理动态分配的对象。当一个对象应该被释放时，指向它的智能指针可以确保自动地释放它。